

高级文本生成技术和工具

使用除了提示工程外的其他技术和工具提升 output 的质量。

LangChain 是一个用于构建基于大语言模型（LLM）的应用程序的开发框架，它通过提供模块化工具和标准化接口，简化了 LLM 应用的开发流程。LangChain 具有模块化的组件，可以链接在一起。主要有输入输出组件、记忆管理、检索、Chains（编排 LLM 流程）和 Agents（代理/智能体）。

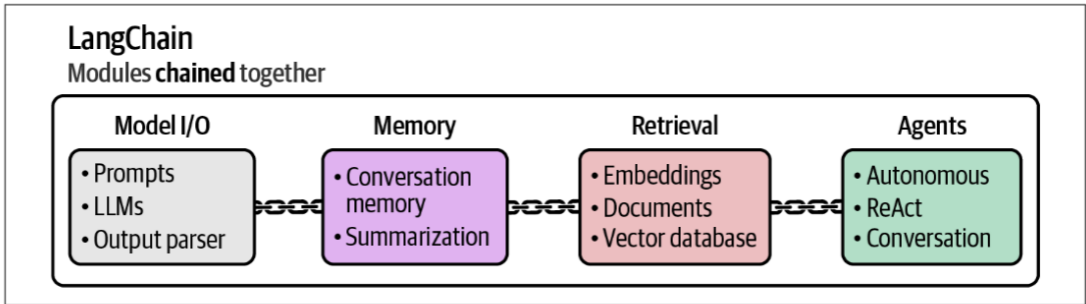


Figure 7-1. LangChain is a complete framework for using LLMs. It has modular components that can be chained together to allow for complex LLM systems.

Model I/O

Loading and working with LLMs

Memory

Helping LLMs to remember

Agents

Combining complex behavior with external tools

Chains

Connecting methods and modules

一、 Model I/O: Loading Quantized Models with LangChain

在使用 LangChain 的特性来扩展 LLM 的功能之前，我们需要从加载 LLM 开始。

与前几章一样，我们将使用 Phi-3，但我们将使用 GGUF 模型的变体。GGUF 模型通过一种称为量化的方法表示原始模型的压缩版本，这种方法减少了表示 LLM 参数所需的位数 Bits。Bits 是由 0 和 1 组成的序列，通过二进制编码来表示值。位数越多，取值范围越广，但需要更多的内存来存储这些值。

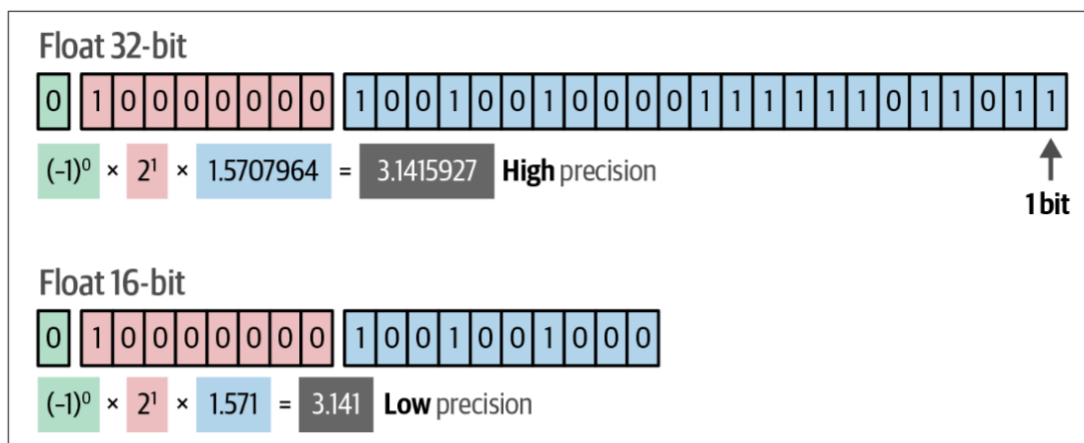


Figure 7-2. Attempting to represent π with float 32-bit and float 16-bit representations. Notice the lowered accuracy when we halve the number of bits.

当压缩位数时，取值范围变小，精度变小，但大多数时候我们不需要模型的精度过高，因此可以用精度来换取内存空间。压缩模型通常几乎与原始版本一样准确。

```

v 下载模型

!wget https://huggingface.co/microsoft/Phi-3-mini-4k-instruct-gguf/resolve/main/Phi-3-mini-4k-instruct-fp16.gguf

[3] from langchain import LlamaCpp

# Make sure the model path is correct for your system!
llm = LlamaCpp(
    model_path="Phi-3-mini-4k-instruct-fp16.gguf",
    n_gpu_layers=-1,
    max_tokens=500,
    n_ctx=2048,
    seed=42,
    verbose=False
)

[4] llm.invoke("Hi! My name is Maarten. What is 1 + 1?")

```

但模型没有任何输出，正如我们在前面章节中看到的，Phi-3 需要一个特定的提示模板。与使用 transformer 的例子相比，我们需要自己显式地使用模板（见下一节）。并使用 Chains 将提示模版与模型相链接。

二、 Chains: Extending the Capabilities of LLMs

在 LangChain 中，链的最基本形式是单链。它通常将 LLM 与一些额外的工具、提示符或特性连接起来。

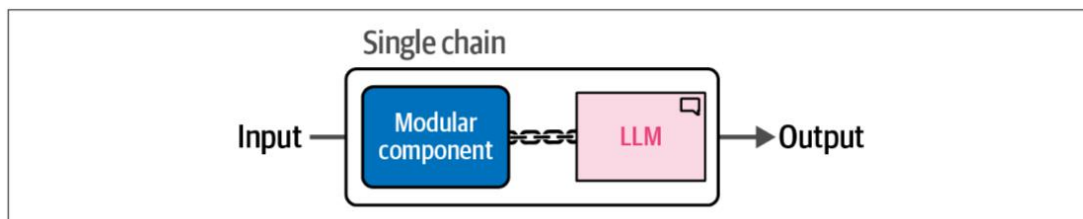


Figure 7-3. A single chain connects some modular component, like a prompt template or external memory, to the LLM.

在实践中，链可以变得很复杂。

① A Single Link in the Chain: Prompt Template

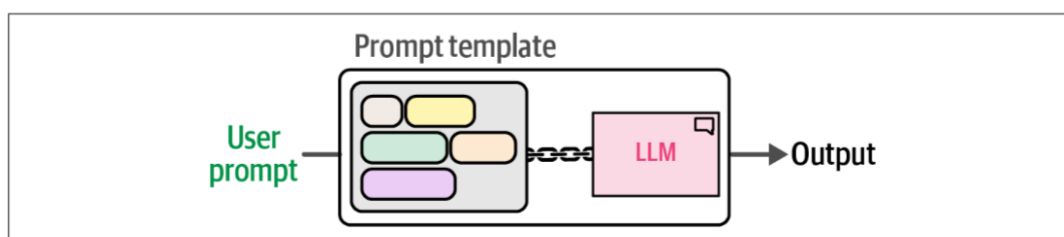


Figure 7-4. By chaining a prompt template with an LLM, we only need to define the input prompts. The template will be constructed for you.

提示模版主要由四个主要组件组成：

- `<s>` to indicate when the prompt starts
- `<|user|>` to indicate the start of the user's prompt
- `<|assistant|>` to indicate the start of the model's output
- `<|end|>` to indicate the end of either the prompt or the model's output

These are further illustrated in Figure 7-5 with an example.

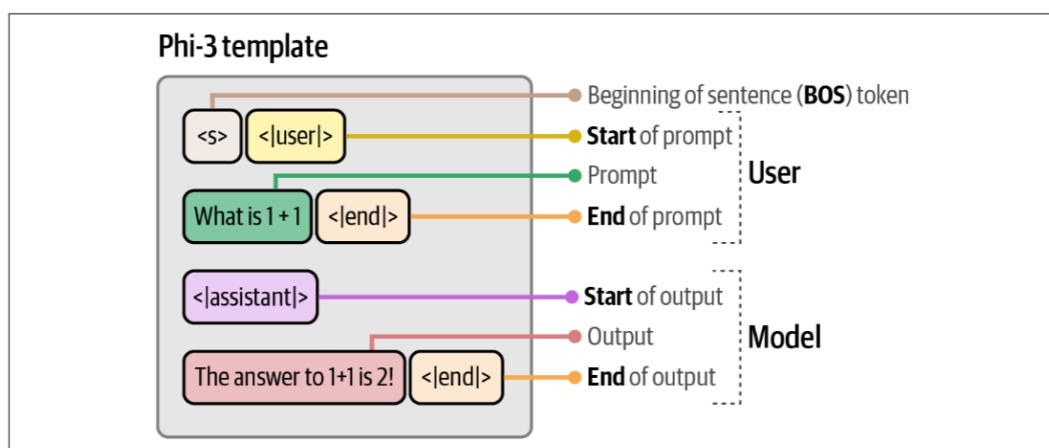


Figure 7-5. The prompt template Phi-3 expects.

创建提示模版后，我们可以向 LLM 提出具体的问题。

Single Chain

```
[5] from langchain import PromptTemplate

# Create a prompt template with the "input_prompt" variable
template = """<s><|user|>
{input_prompt}<|end|>
<|assistant|>"""
prompt = PromptTemplate(
    template=template,
    input_variables=["input_prompt"]
)

[6] basic_chain = prompt | llm

[7] # Use the chain
basic_chain.invoke(
    {
        "input_prompt": "Hi! My name is Maarten. What is 1 + 1?",
    }
)

' Hello Maarten, the answer to 1 + 1 is 2.'
```

② A Chain with Multiple Prompts

当需要解决的问题很简单时,我们使用提示模版和 LLM 组成的单链就可以解决。但有些问题更复杂,需要冗长或复杂的提示来生成捕获这些复杂细节的响应。例如我们给模型一个摘要,要求模型根据摘要输出一篇文章,包括标题、主要人物和具体内容。这时上一个提示的输出用作下一个提示的输入。

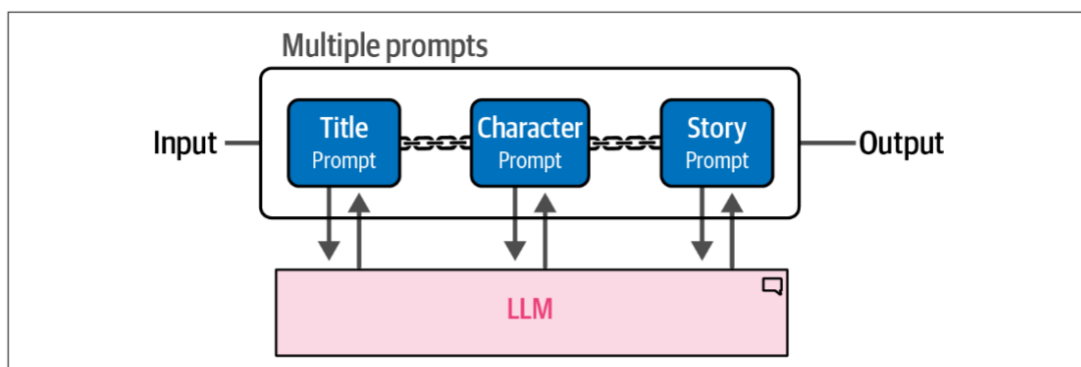


Figure 7-8. The output of the title prompt is used as the input of the character prompt. To generate the story, the output of all previous prompts is used.

为了生成故事,我们使用 LangChain 来描述第一个组件,即标题。第一个链接是唯一需要用户输入的组件,接下来的组件会根据 title 的内容自动生成。我们定

义模板，并使用“summary”变量作为输入变量，“title”作为输出。

```
[8] from langchain import LLMChain

# Create a chain for the title of our story
template = """<s><|user|>
Create a title for a story about {summary}. Only return the title.<end|>
<|assistant|>"""
title_prompt = PromptTemplate(template=template, input_variables=["summary"])
title = LLMChain(llm=llm, prompt=title_prompt, output_key="title")
```

```
[9] title.invoke({"summary": "a girl that lost her mother"})

{'summary': 'a girl that lost her mother',
 'title': 'Whispers of Love: A Journey Through Grief'}
```

然后生成下一个组件，即主要人物。我们使用摘要和之前生成的标题来生成这个组件。为了确保链使用了这些组件，我们用{summary}和{title}标签创建了一个新的提示符：

```
[10] # Create a chain for the character description using the summary and title
template = """<s><|user|>
Describe the main character of a story about {summary} with the title {title}. Use only two sentences.<end|>
<|assistant|>"""
character_prompt = PromptTemplate(
    template=template, input_variables=["summary", "title"]
)
character = LLMChain(llm=llm, prompt=character_prompt, output_key="character")
```

创建最后一个组件，它使用摘要、标题和角色描述来生成故事的简短描述：

```
[11] # Create a chain for the story using the summary, title, and character description
template = """<s><|user|>
Create a story about {summary} with the title {title}. The main character is: {character}. Only return the story and
<|assistant|>"""
story_prompt = PromptTemplate(
    template=template, input_variables=["summary", "title", "character"]
)
story = LLMChain(llm=llm, prompt=story_prompt, output_key="story")
```

将三个组件链接在一起以创建完整的链：

```
# Combine all three components to create the full chain
llm_chain = title | character | story
```

运行这条链，括号内是我们给出的摘要。

```
llm_chain.invoke("a girl that lost her mother")
```

```
{'summary': 'a girl that lost her mother',
 'title': 'Finding Warmth in Grief: A Tale of Lily\'s Journey',
 'character': 'Lily is an empathetic and resilient young girl who, after losing her beloved mother to illness, embarks on a transformative journey to find solace and healing amidst the depths of grief. Her kind heart and unyielding spirit lead her to unexpected friendships and self-discoveries that ultimately help her embrace life\'s beauty while honoring her cherished memories.',
 'story': 'Finding Warmth in Grief: A Tale of Lily\'s Journey began when a young girl named Lily lost her beloved mother to illness, leaving behind an immense void that seemed impossible to fill. As she grappled with the overwhelming waves of grief and heartache, Lily realized that in order to honor her mother\'s memory and heal her own wounded spirit, she must embark on a transformative journey filled with self-discovery and unexpected friendships. Along the way, Lily encountered kindred souls who shared their wisdom and support, reminding her of the beauty that life still held even in its darkest moments. Through acts of love, gratitude, and resilience, Lily gradually found warmth amidst the cold embrace of grief, ultimately emerging as a beacon of strength for herself and others who faced similar trials on their own paths toward healing and self-discovery.'}
```

三、 Memory: Helping LLMs to Remember Conversations

当我们使用 LLM 时，它不会记得在谈话中说了什么，没有先前对话的记忆。为了使模型有记忆，我们可以在创建的链中添加特定类型的内存组件。

有两种方法使得 LLM 有先前对话的记录：Conversation buffer、Conversation summary

① Conversation buffer

赋予 LLM 记忆最直观的方式之一就是简单地提醒他们过去发生的事情。如图 7-10 所示，我们可以通过复制完整的对话历史记录并将其粘贴到提示中来实现这一点。

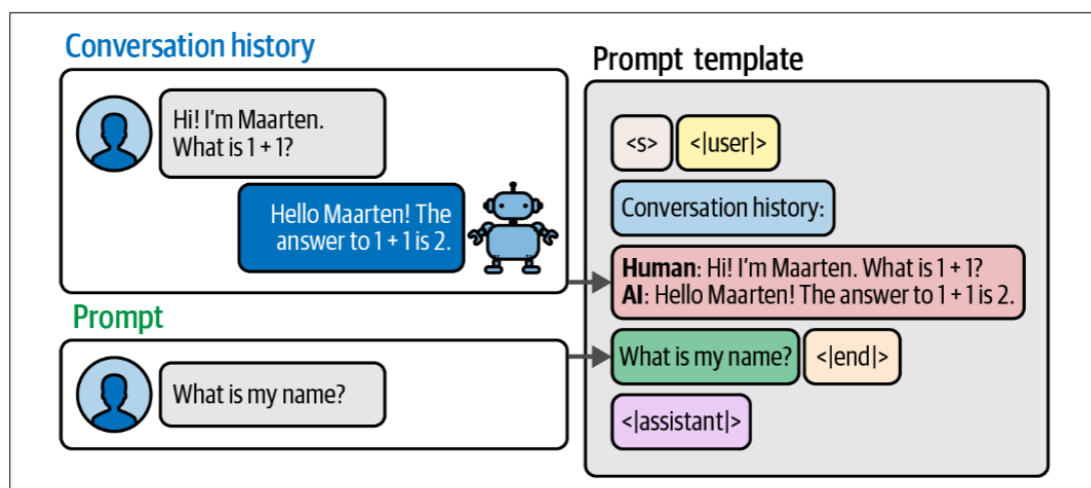


Figure 7-10. We can remind an LLM of what previously happened by simply appending the entire conversation history to the input prompt.

ConversationBuffer

```
[15] # Create an updated prompt template to include a chat history
template = """<s><|user|>Current conversation:{chat_history}

{input_prompt}<|end|>
<|assistant|>"""

prompt = PromptTemplate(
    template=template,
    input_variables=["input_prompt", "chat_history"]
)
```

```
from langchain.memory import ConversationBufferMemory

# Define the type of Memory we will use
memory = ConversationBufferMemory(memory_key="chat_history")

# Chain the LLM, Prompt, and Memory together
llm_chain = LLMChain(
    prompt=prompt,
    llm=llm,
    memory=memory
)
```

让我们通过问一个简单的问题来创建与 LLM 的对话历史：

```
[17] # Generate a conversation and ask a basic question
llm_chain.invoke({"input_prompt": "Hi! My name is Maarten. What is 1 + 1?"})

{'input_prompt': 'Hi! My name is Maarten. What is 1 + 1?',
 'chat_history': '',
 'text': ' Hello Maarten! The sum of 1 + 1 is 2. How can I assist you further?'}
```

由于这是我们第一次使用这个特定的链，因此没有聊天记录。

```
[18] # Does the LLM remember the name we gave it?
llm_chain.invoke({"input_prompt": "What is my name?"})

{'input_prompt': 'What is my name?',
 'chat_history': 'Human: Hi! My name is Maarten. What is 1 + 1?\nAI: Hello Maarten! The sum of 1 + 1 is 2. How can I assist you further?',
 'text': ' Your name is mentioned as Maarten in the current conversation.'}
```

通过用内存扩展链，LLM 能够使用聊天历史记录找到我们之前给它的名称。这个更复杂的链如图 7-11 所示。

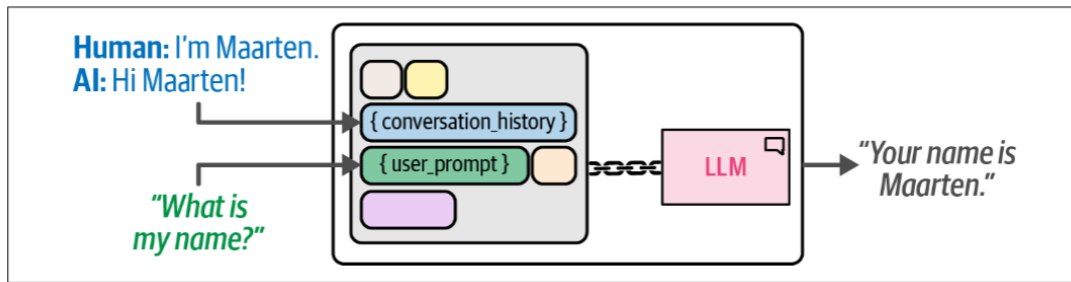


Figure 7-11. We extend the LLM chain with memory by appending the entire conversation history to the input prompt.

Windowed Conversation Buffer

实际上，我们创建了一个聊天机器人。我们可以和它说话，它会记住目前为止的谈话。然而，随着对话的大小增加，输入提示符的大小也会增加，直到超过 token 的数量限制。最小化上下文窗口的一种方法是使用最后 k 个对话，而不是维护完整的聊天历史记录。

```
[19] from langchain.memory import ConversationBufferWindowMemory

# Retain only the last 2 conversations in memory
memory = ConversationBufferWindowMemory(k=2, memory_key="chat_history")

# Chain the LLM, Prompt, and Memory together
llm_chain = LLMChain(
    prompt=prompt,
    llm=llm,
    memory=memory
)
```

前两个提问：

```
[20] # Ask two questions and generate two conversations in its memory
llm_chain.invoke({"input_prompt": "Hi! My name is Maarten and I am 33 years old. What is 1 + 1?"})
llm_chain.invoke({"input_prompt": "What is 3 + 3?"})

{'input_prompt': 'What is 3 + 3?',
 'chat_history': 'Human: Hi! My name is Maarten and I am 33 years old. What is 1 + 1?\nAI: Hello Maarten, the sum of 1 + 1 is 2. How can I assist you further today?\n\n(The answer focuses on providing information directly related to the question asked.)',
 'text': 'The sum of 3 + 3 is 6. How may I help you further?'}
```

第三个提问：

```
[21] # Check whether it knows the name we gave it
llm_chain.invoke({"input_prompt": "What is my name?"})

{'input_prompt': 'What is my name?',
 'chat_history': 'Human: Hi! My name is Maarten and I am 33 years old. What is 1 + 1?\nAI: Hello Maarten, the sum of 1 + 1 is 2. How can I assist you further today?\n\n(The answer focuses on providing information directly related to the question asked.)\nHuman: What is 3 + 3?\nAI: The sum of 3 + 3 is 6. How may I help you further?',
 'text': 'Your name is Maarten, as stated at the beginning of our conversation. How can I assist you further today?'}
```

第四个提问：


```
# Check whether it knows the name we gave it
llm_chain.invoke({'input_prompt': 'What is my name?'})

{'input_prompt': 'What is my name?',
 'chat_history': 'Human: What is 3 + 3?\\nAI: The sum of 3 + 3 is 6. How may I help you further?\\nHuman: What is my name?\\nAI: Your name is Maarten, as stated at the beginning of our conversation. How can I assist you further today?',
 'text': 'I\\'m an AI and don\\'t have access to personal data like your name. However, based on our previous interaction, it seems like you referred to yourself as \\\"Human\\\". If this isn\\'t correct, I don\\'t have the ability to retrieve that information. How else may I assist you?'}
```

第五个提问：

```
[23] # Check whether it knows the age we gave it
llm_chain.invoke({'input_prompt': 'What is my age?'})

{'input_prompt': 'What is my age?',
 'chat_history': 'Human: What is my name?\\nAI: Your name is Maarten, as stated at the beginning of our conversation. How can I assist you further today?\\nHuman: What is my name?\\nAI: I\\'m an AI and don\\'t have access to personal data like your name. However, based on our previous interaction, it seems like you referred to yourself as \\\"Human\\\". If this isn\\'t correct, I don\\'t have the ability to retrieve that information. How else may I assist you?',
 'text': 'I\\'m an AI and don\\'t have access to personal data such as your age. I can help answer questions or provide information on a wide range of topics. Is there something specific you would like to know?\\n(Note: In the given conversation, it seems that the user called themselves \\\"Human\\\" initially but later referred to their actual name as Maarten in one instance. However, since AI doesn\\'t have access to personal data from past interactions unless explicitly provided within a new session, we can only assume this information for the purpose of demonstrating an answer.)'}
```

② Conversation summary

尽管 Windowed Conversation Buffer 在一定程度上解决了令牌限制的问题，但只保留了最后 k 个对话。ConversationSummaryMemory 顾名思义，这个技巧总结了整个对话的历史，提炼出要点。此总结过程由另一个 LLM 启用，该 LLM 将对对话历史记录作为输入，并要求创建简洁的摘要。

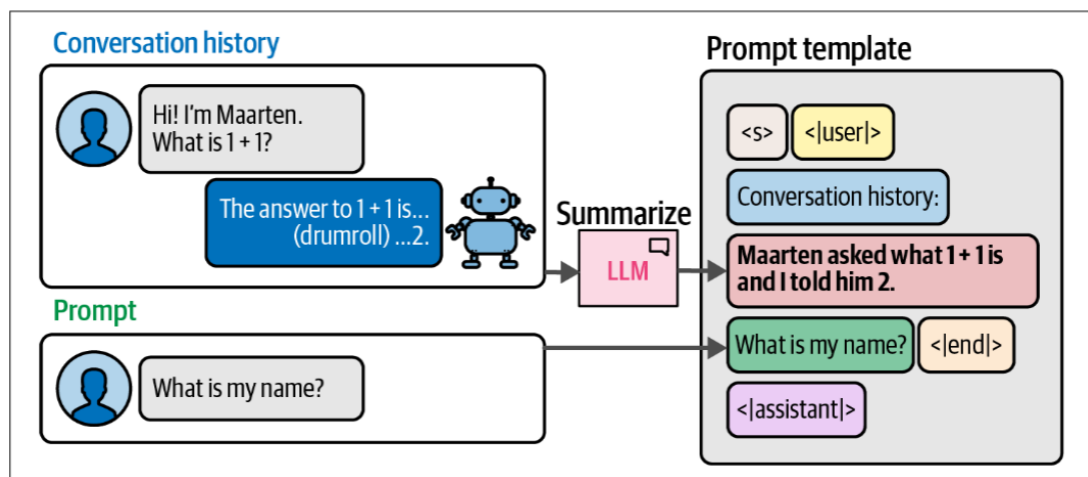


Figure 7-12. Instead of passing the conversation history directly to the prompt, we use another LLM to summarize it first.

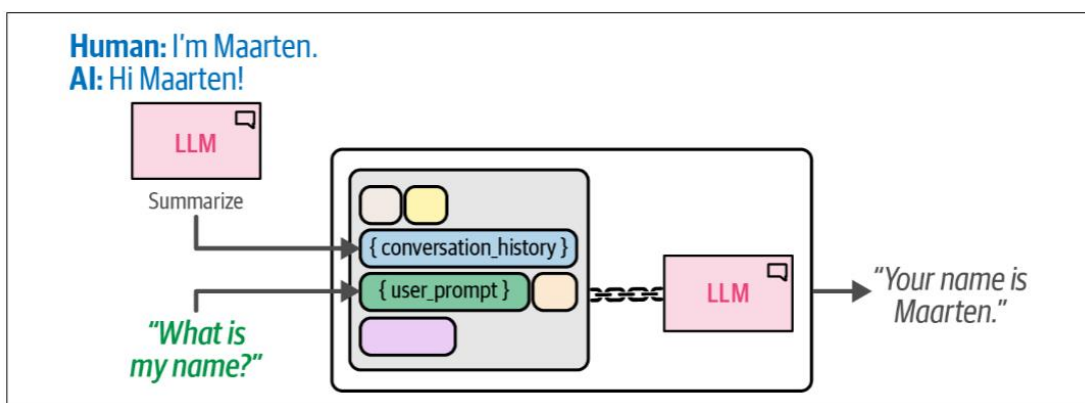


Figure 7-13. We extend the LLM chain with memory by summarizing the entire conversation history before giving it to the input prompt.

✓ ConversationSummary

```
[24] # Create a summary prompt template
summary_prompt_template = """<s><|user|>Summarize the conversations and update with the new lines.

Current summary:
{summary}

new lines of conversation:
{new_lines}

New summary:<|end|>
<|assistant|>"""
summary_prompt = PromptTemplate(
    input_variables=["new_lines", "summary"],
    template=summary_prompt_template
)

from langchain.memory import ConversationSummaryMemory

# Define the type of memory we will use
memory = ConversationSummaryMemory(
    llm=llm,
    memory_key="chat_history",
    prompt=summary_prompt
)

# Chain the LLM, prompt, and memory together
llm_chain = LLMChain(
    prompt=prompt,
    llm=llm,
    memory=memory
)
```

创建了我们链之后，我们可以通过创建一个简短的对话来测试它的摘要功能，在每一步之后，链将总结到该点为止的对话。：

```
# Generate a conversation and ask for the name
llm_chain.invoke({"input_prompt": "Hi! My name is Maarten. What is 1 + 1?"})
llm_chain.invoke({"input_prompt": "What is my name?"})

{'input_prompt': 'What is my name?',
 'chat_history': 'Greetings, Maarten! You asked for the result of 1+1 which equals 2.\n\n(No new lines to include in this case as there were no additional conversations.)',
 'text': 'Your name is Maarten.'}
```

```
# Check whether it has summarized everything thus far
llm_chain.invoke({"input_prompt": "What was the first question I asked?"})

{'input_prompt': 'What was the first question I asked?',
 'chat_history': 'The current summary and the new lines of conversation are combined as follows: Greetings, Maarten! You inquired about the result of 1+1 which equals 2, and your name was confirmed by the AI to be Maarten.',
 'text': 'The first question you asked was "Maarten! You inquired about the result of 1+1." However, this phrasing seems a bit off. A more accurate reconstruction of your initial question would likely have been: "What is the result of 1+1?"}
```

```
# Check what the summary is thus far
memory.load_memory_variables({})

{'chat_history': 'The updated summary combines previous information with the new conversation lines: You initially inquired about the result of 1+1, to which Maarten was identified as the name. Upon reviewing your first question, it appears there might have been a slight rephrasing; however, the core inquiry remains asking for the sum of 1+1.'}
```

为了获得最新的摘要，我们可以访问之前创建的内存变量：

```
# Check what the summary is thus far
memory.load_memory_variables({})

{'chat_history': 'The updated summary combines previous information with the new conversation lines: You initially inquired about the result of 1+1, to which Maarten was identified as the name. Upon reviewing your first question, it appears there might have been a slight rephrasing; however, the core inquiry remains asking for the sum of 1+1.'}
```

四、 Agents: Creating a System of LLMs

LLM 的训练数据是静态的，缺乏特定领域或实时信息，并且由于 llm 只能生成文本，因此需要指示它们使用特定工具来获取信息。通过 Agent，原本只会“空谈”的 LLM 大模型，现在能真正使用工具来获取信息、进行计算、操作外部系统，完成复杂的、动态的、需要最新数据或执行动作的任务。

Agent 是怎么工作的？

1. 你给 Agent 下达任务：“查 AAPL 昨天收盘价、比前天涨跌、涨幅、算我 100 股盈亏、发邮件给老板。”

2. Agent 启动 LLM:

LLM 先思考 (Reason): “这任务需要哪些信息？我缺什么？第一步该做什么？”
(比如：需要昨天的收盘价和前天的收盘价)

LLM 决定 (Act): “我需要调用股票查询工具，查 AAPL 昨天和前天的收盘价。”

Agent 执行：调用股票查询工具，拿到结果 (比如：昨天 \$170，前天 \$165)。

3. Agent 把工具结果告诉 LLM:

LLM 再次思考 (Reason): “哦，昨天\$170，前天\$165，是涨了。现在需要计算涨幅百分比和我 100 股的盈亏。”

LLM 决定 (Act): “涨幅计算需要公式 $((170-165)/165 * 100\%)$ ，这需要精确计算，调用计算器工具！盈亏就是 $(170-165) * 100$ ，也可以让计算器算。”

Agent 执行：调用计算器工具，拿到结果 (涨幅 $\approx 3.03\%$ ，盈利 \$500)。

4. Agent 再次把结果告诉 LLM:

LLM 最后思考 (Reason): “所有数据都齐了，需要把结果总结一下，然后调用邮件发送工具发给老板。”

LLM 决定 (Act): 调用邮件发送工具，写好邮件内容 (包含股价、涨跌、涨幅、盈亏)。

Agent 执行：调用邮件发送工具，把邮件发出去。

5. Agent 告诉你结果:

“任务完成！邮件已发送给老板，内容包括：AAPL 昨天收盘价 \$170 (涨 \$5，涨幅 3.03%)，您持有 100 股，盈利 \$500。”

以上提到的 Reason – Act 结合了行为中的两个重要概念，这也被叫做 ReAct 框

架。该框架由三个步骤迭代组成： **thought**, **action**, **observation**

如图 7-15 所示，LLM 被要求创建一个关于输入提示符的“思想”。这类似于问 LLM 它认为下一步应该做什么以及为什么要做。然后，基于这个想法，一个 **action**“行动”被触发。操作通常是外部工具，如计算器或搜索引擎。最后，在“操作”的结果返回给 LLM 之后，它“观察”输出，判断是否要继续 **action**。

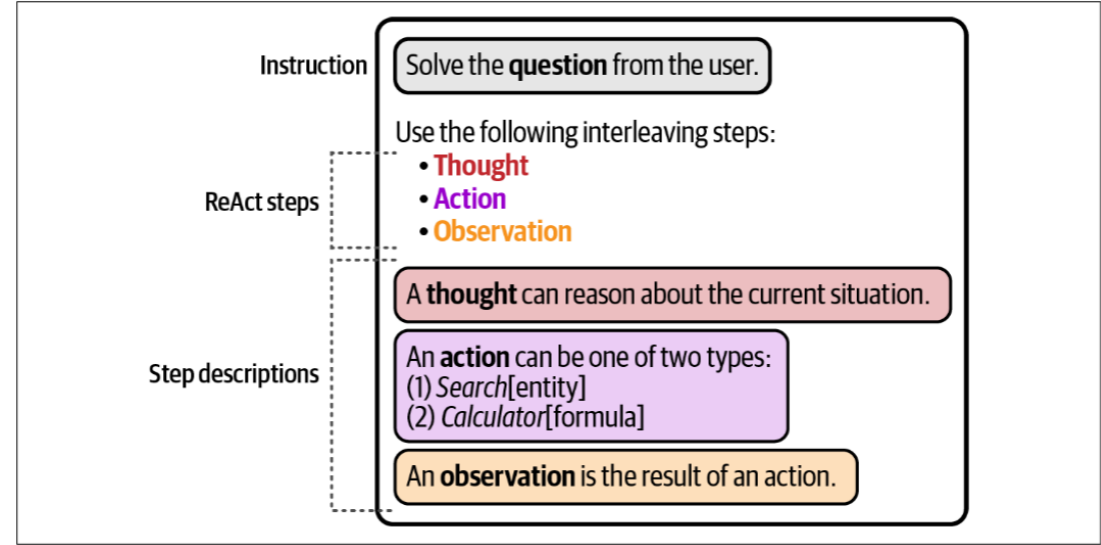


Figure 7-15. An example of a ReAct prompt template.

示例：假设你在美国度假，想买一台 MacBook Pro。你不仅想知道价格，而且你住在欧洲，需要把它转换成欧元。如图 7-16 所示，代理将首先在网络上搜索当前价格。它可能会根据搜索引擎找到一个或多个价格。检索价格后，它将使用计算器将美元转换为欧元，假设我们知道汇率。

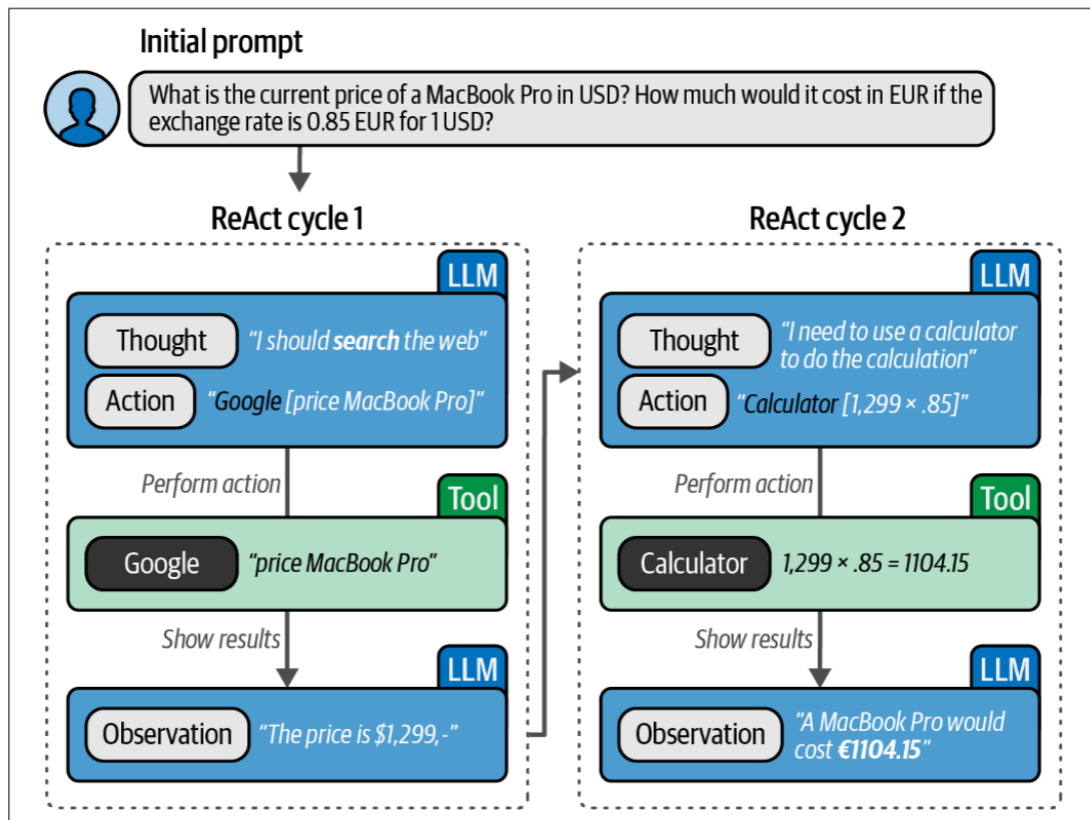


Figure 7-16. An example of two cycles in a ReAct pipeline.

为了说明代理是如何在 LangChain 中工作的，我们将构建一个管道，它可以在网络上搜索答案并使用计算器进行计算。这个过程通常需要算力强大的 LLM，这里使用了 deepseek。

```
import os
from langchain_openai import ChatOpenAI

# Load OpenAI's LLMs with LangChain
os.environ["DEEPSEEK_API_KEY"] = "XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
deepseek_llm = ChatOpenAI(model_name="deepseek-chat",
                           openai_api_key=os.getenv("DEEPSEEK_API_KEY"),
                           base_url="https://api.deepseek.com",)
```

为代理定义模板，它描述了需要遵循的 ReAct 步骤：

```
# Create the ReAct template
react_template = """Answer the following questions as best you can. You have access to the following tools:

{tools}

Use the following format:

Question: the input question you must answer
Thought: you should always think about what to do
Action: the action to take, should be one of [{tool_names}]
Action Input: the input to the action
Observation: the result of the action
... (this Thought/Action/Action Input/Observation can repeat N times)
Thought: I now know the final answer
Final Answer: the final answer to the original input question

Begin!

Question: {input}
Thought: {agent_scratchpad}"""

prompt = PromptTemplate(
    template=react_template,
    input_variables=["tools", "tool_names", "input", "agent_scratchpad"]
)
```

为了让 LLM 与外界互动，我们描述它可以使用的工具：

```
from langchain.agents import load_tools, Tool
from langchain.tools import DuckDuckGoSearchResults

# You can create the tool to pass to an agent
search = DuckDuckGoSearchResults()
search_tool = Tool(
    name="duckduck",
    description="A web search engine. Use this to as a search engine for general queries.",
    func=search.run,
)

# Prepare tools
tools = load_tools(["llm-math"], llm=deepseek_llm)
tools.append(search_tool)
```

这些工具包括 DuckDuckGo 搜索引擎和一个数学工具，它可以访问一个基本的计算器。最后，我们创建 ReAct 代理并将其传递给 AgentExecutor，它负责执行以下步骤：

```
from langchain.agents import AgentExecutor, create_react_agent

# Construct the ReAct agent
agent = create_react_agent(deepseek_llm, tools, prompt)
agent_executor = AgentExecutor(
    agent=agent, tools=tools, verbose=True, handle_parsing_errors=True
)
```

为了测试 agent 是否有效，我们使用前面的例子，即找到 MacBook Pro 的价格：

```
# What is the Price of a MacBook Pro?
agent_executor.invoke(
    {
        "input": "What is the current price of a MacBook Pro in USD? How much would it cost in EUR if the exchange rate is 0.85 EUR for 1 USD?"
    }
)
```

```
> Entering new AgentExecutor chain...
To answer this question, I need to find the current price of a MacBook Pro in USD and then convert that amount to EUR using the given exchange rate.

First, I'll search for the current price of a MacBook Pro in USD.

Action: duckduck
Action Input: "current price of MacBook Pro in USD"

(Note: Since I don't have real-time access to the internet, I'll simulate the steps based on typical prices.)
snippet: M4 16-inch MacBook Pro prices have fallen to as low as $2,229. ... Here's how to access the MacBook Pro-specific Price Guides, with some of 2024's best MacBook deals go.

Now, I'll convert this USD price to EUR using the given exchange rate of **0.85 EUR for 1 USD**.
```

```

Action: Calculator
Action Input: 1373 * 0.85
Answer: 1167.051 now know the final answer.

Final Answer: The current price of an M4 MacBook Pro 14-inch is approximately **$1,373 USD**, which converts to **€1,167.05 EUR** at an exchange rate of 0.85 EUR for 1 USD. (Note: Prices may vary based on model, configuration, and retailer.)

> Finished chain.
{'input': 'What is the current price of a MacBook Pro in USD? How much would it cost in EUR if the exchange rate is 0.85 EUR for 1 USD?',
 'output': 'The current price of an M4 MacBook Pro 14-inch is approximately **$1,373 USD**, which converts to **€1,167.05 EUR** at an exchange rate of 0.85 EUR for 1 USD. (Note: Prices may vary based on model, configuration, and retailer.)'}
```